

LECTURE 7

Convolutional Neural Networks

Filters, feature maps, pooling and the architectures that taught machines to see — from pixels to target recognition
Case study: SAR target recognition with a 93,962-parameter CNN

AK DHAMIJA, Researcher & Instructor

AI & Machine Learning

dhamija.ak@gmail.com • 9811453163

Learning Objectives & Session Structure

By the end of this lecture you will be able to

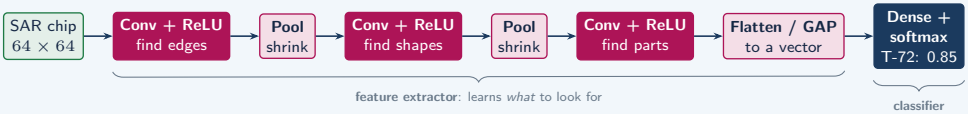
1. Explain why fully-connected networks are a poor fit for images
2. Compute a convolution, ReLU and max-pooling by hand on a 6×6 image
3. Use the output-size formula with padding and stride
4. Count the parameters of a convolutional layer and a full CNN
5. Explain receptive fields and the edge $\rightarrow$ part $\rightarrow$ object hierarchy
6. Describe LeNet, AlexNet, VGG and ResNet, and transfer learning
7. Assess CNNs for SAR / EO target recognition, including their vulnerabilities

Session Structure (45 min)

- **5 min** — Opening Q&A: hidden layers and backpropagation (L6)
- **12 min** — Images as numbers; convolution, ReLU, pooling: worked example
- **8 min** — Padding, stride, channels, parameters; a complete CNN
- **10 min** — Architectures, transfer learning, defence uses and failure modes
- **10 min** — Auto-quiz (15 questions)

Pre-requisite: Lecture 6 (hidden layers and backpropagation by hand). A CNN is trained exactly the same way — only the layers are new.

The Big Picture in One Slide



Convolution

Slide a small learned filter over the image; it lights up wherever its pattern appears.

Pooling

Keep the strongest response in each small region: smaller maps, tolerance to small shifts.

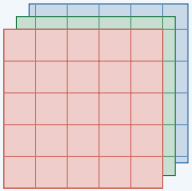
Depth

Early layers see edges; deeper layers combine them into tracks, turrets, whole vehicles.

Images Are Grids of Numbers

0	0	0	0	0	0
0	0	0	0	0	0
0	0	9	9	9	9
0	0	9	9	9	9
0	0	9	9	9	9
0	0	9	9	9	9

grayscale: one number per pixel (0 = dark, 9 = bright)



colour: 3 channels (R, G, B) stacked

Shapes

A colour image is a 3-D array: height $\times$ width $\times$ channels. A 224×224 RGB photo holds $224 \cdot 224 \cdot 3 = 150\,528$ numbers.

SAR and IR imagery are usually single-channel; multispectral satellite imagery may have 4–13 channels. CNNs handle any number.

Why Not a Plain Fully-Connected Network?

1. Too many weights. Connecting a $224 \times 224 \times 3$ image to just 1 000 hidden units needs $150\,528 \times 1\,000 \approx$ **150 million** weights — for the first layer alone.

2. Ignores neighbourhoods. Flattening the image into a list throws away which pixels are next to each other — yet edges, corners and textures are all *local*.

3. No shift tolerance. A tank in the top-left corner and the same tank in the centre activate completely different weights; the network must learn "tank" separately at every position.

The CNN answer

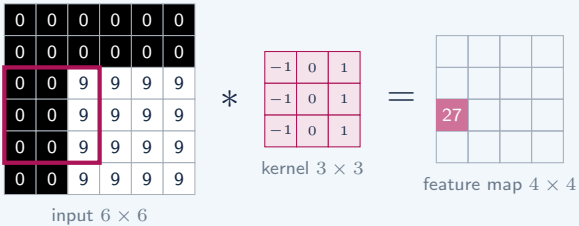
- **Local connectivity**: each neuron looks at a small patch (e.g. 3×3)
- **Weight sharing**: the same filter is reused at every position
- **Hierarchy**: stack layers so patches grow into whole objects

Result for the same first layer

64 filters of size $3 \times 3 \times 3$ need only $(3 \cdot 3 \cdot 3 + 1) \times 64 =$ **1 792** parameters — about $84\,000\times$ fewer — and they detect a pattern anywhere in the image.

Inspiration: Hubel and Wiesel's studies of the visual cortex (1959–62) found neurons responding to edges at particular orientations in small regions of the visual field.

Convolution: A Template Sliding Over the Image



2-D CONVOLUTION (AS USED IN CNNS)

$$y_{i,j} = \sum_{m=0}^{k-1} \sum_{n=0}^{k-1} K_{m,n} \; x_{i+m,\, j+n} + b$$

Recipe

1. Place the kernel on a 3×3 patch
2. Multiply element by element, add up ($+$ bias)
3. Write the result into the output map
4. Slide one step and repeat

Analogy: an imagery analyst sweeping a magnifier with a stencil across a photo, noting how well each spot matches the stencil.

Strictly this is *cross-correlation* (the kernel is not flipped). Deep-learning libraries all call it "convolution"; since kernels are learned, the difference does not matter.

Worked Example: Set-Up

	0	1	2	3	4	5
0	0	0	0	0	0	0
1	0	0	0	0	0	0
2	0	0	9	9	9	9
3	0	0	9	9	9	9
4	0	0	9	9	9	9
5	0	0	9	9	9	9

Input X (6×6): the corner of a bright object (e.g. a vehicle return) on a dark background.

Kernel K_1: left-edge detector

$$\begin{pmatrix} -1 & 0 & 1 \\ -1 & 0 & 1 \\ -1 & 0 & 1 \end{pmatrix}$$

Positive where brightness *increases* left $\rightarrow$ right.

Kernel K_2: top-edge detector

$$\begin{pmatrix} -1 & -1 & -1 \\ 0 & 0 & 0 \\ 1 & 1 & 1 \end{pmatrix}$$

Positive where brightness *increases* top $\rightarrow$ bottom.

Plan

- Convolve X with each kernel (stride 1, no padding, bias 0) $\rightarrow$ two 4×4 feature maps
- Apply ReLU: $\max(0, \cdot)$
- Apply 2×2 max-pooling with stride 2 $\rightarrow$ two 2×2 maps

These hand-designed kernels are classical (Prewitt) edge filters. A trained CNN *learns* kernels like these in its first layer — nobody programs them.

Output position (0,0) with K_1

Patch = rows 0–2, columns 0–2:

$$\begin{pmatrix} 0 & 0 & 0 \\ 0 & 0 & 0 \\ 0 & 0 & 9 \end{pmatrix} \odot \begin{pmatrix} -1 & 0 & 1 \\ -1 & 0 & 1 \\ -1 & 0 & 1 \end{pmatrix} \rightarrow 9 \cdot 1 = \mathbf{9}$$

Only the bottom-right pixel is bright, and it meets a $+1$.

Output position (2,0) with K_1

Patch = rows 2–4, columns 0–2:

$$\begin{pmatrix} 0 & 0 & 9 \\ 0 & 0 & 9 \\ 0 & 0 & 9 \end{pmatrix} \odot \begin{pmatrix} -1 & 0 & 1 \\ -1 & 0 & 1 \\ -1 & 0 & 1 \end{pmatrix} \rightarrow 9 + 9 + 9 = \mathbf{27}$$

A full vertical edge fills the right column: maximum response.

Output position (2,2) with K_1

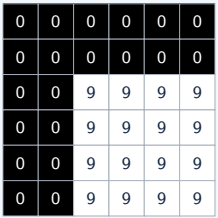
Patch = rows 2–4, columns 2–4 (all bright):

$$\begin{pmatrix} 9 & 9 & 9 \\ 9 & 9 & 9 \\ 9 & 9 & 9 \end{pmatrix} \odot K_1 \rightarrow (-27) + 0 + 27 = \mathbf{0}$$

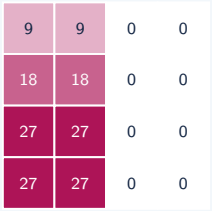
Key insight: a uniform region gives 0 — the left and right columns cancel. An edge kernel responds only where brightness *changes*.

Each output value is a dot product between the kernel and a patch — the same "similarity score" idea as attention in Lectures 8–9 (transformers).

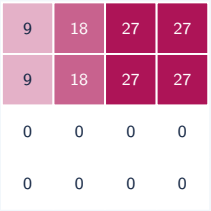
Worked Example — Step 2: The Full Feature Maps



input X



$X * K_1$ (left edge)



$X * K_2$ (top edge)

Reading the maps

K_1 fires down the *left side* of the object (columns 0–1); K_2 fires along its *top side* (rows 0–1). Values ramp up $9 \to 18 \to 27$ as more of the edge falls inside the 3×3 window.

Output size: a 3×3 kernel fits in $6 - 3 + 1 = 4$ positions per direction, so $6 \times 6 \to 4 \times 4$. Each kernel produces one *feature map*; a layer with 32 kernels produces 32 maps.

RELU

$$\text{ReLU}(z) \quad = \quad \max(0,\, z)$$

What if the edge runs the other way?

Use the mirrored kernel (a *right*-edge detector) on the same image; every response flips sign:

$$-K_1 = \begin{pmatrix} 1 & 0 & -1 \\ 1 & 0 & -1 \\ 1 & 0 & -1 \end{pmatrix}$$

-9	-9	0	0
-18	-18	0	0
-27	-27	0	0
-27	-27	0	0

$X * (-K_1)$

ReLU →

0	0	0	0
0	0	0	0
0	0	0	0
0	0	0	0

all zeros

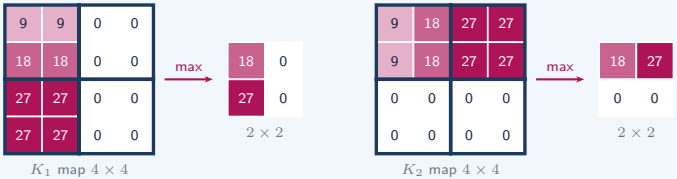
Interpretation: "there is no right-hand edge here". ReLU keeps evidence *for* a pattern and discards evidence against it, so each map answers one clear question.

Why a non-linearity at all?

Without it, stacked convolutions collapse into a single convolution — a deep network would be no more powerful than one layer.

Our K_1 and K_2 maps are already non-negative, so ReLU leaves them unchanged. ReLU's gradient is exactly 1 for positive inputs — no shrinking, unlike the sigmoid's ≤ 0.25.

Worked Example — Step 4: Max Pooling (2×2, stride 2)



Top-left block of K_1 map: $\max(9, 9, 18, 18) = 18$; bottom-left: $\max(27, 27, 27, 27) = 27$.

What pooling buys

- $4\times$ **fewer numbers** to process downstream
- **Shift tolerance**: moving the edge by one pixel inside a block leaves the output unchanged
- **Summary**: "left edge in the lower-left quadrant; top edge in the upper-right"

Pooling has **no learned parameters**. Average pooling takes the mean instead of the max.

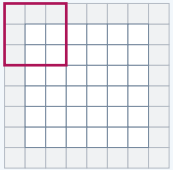
Padding and Stride: Controlling the Output Size

OUTPUT SIZE (PER SPATIAL DIMENSION)

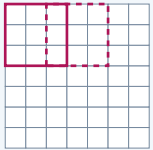
$$n_{\text{out}} = \left\lfloor \frac{n_{\text{in}} + 2p - k}{s} \right\rfloor + 1$$

n = input size, k = kernel size, p = zero-padding, s = stride

Case	n_{in}	k, p, s	Calculation	n_{out}
Our example	6	3,0,1	$(6-3)/1+1$	4
LeNet-5 conv1	32	5,0,1	$(32-5)/1+1$	28
AlexNet conv1	227	11,0,4	$(227-11)/4+1$	55
"Same" padding	224	3,1,1	$(224+2-3)/1+1$	224
Downsampling	224	3,1,2	$\lfloor 223/2 \rfloor + 1$	112



grey ring = padding $p = 1$



stride $s = 2$: jump 2 pixels

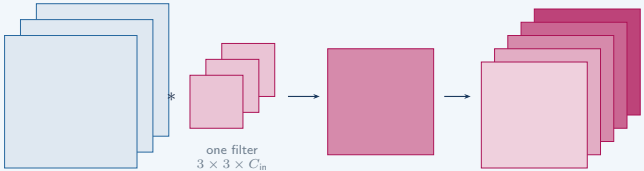
Why pad?

Without padding every layer shrinks the map and edge pixels are under-used. "Same" padding $p = (k-1)/2$ keeps the size unchanged.

Why stride?

Stride 2 halves the map — a learned alternative to pooling, used in many modern networks.

Channels and Filters: Convolution in 3-D



PARAMETERS IN ONE CONVOLUTIONAL LAYER

$$(k \times k \times C_{\text{in}} + 1) \times C_{\text{out}}$$

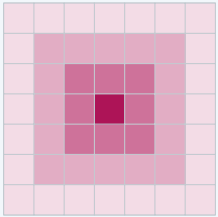
How it works

Each filter spans *all* input channels: it multiplies a 3×3 patch in every channel and adds everything into one number. C_{out} different filters give C_{out} maps.

Example

RGB input, 64 filters of 3×3:
$(3 \cdot 3 \cdot 3 + 1) \times 64 = \mathbf{1\,792}$ parameters.
Next layer, 128 filters on those 64 maps:
$(3 \cdot 3 \cdot 64 + 1) \times 128 = \mathbf{73\,856}$.

The parameter count does *not* depend on image size — a CNN trained on 64×64 chips can be run on a 512×512 scene.



1 layer of 3×3: sees 3×3

2 layers: sees 5×5

3 layers: sees 7×7

Small kernels, stacked

Three 3×3 layers see the same 7×7 area as one 7×7 layer, with $3 \times 9 = 27$ weights instead of 49 per channel pair — and three non-linearities instead of one. This is the VGG design rule.

Layer 1 edges, blobs → **Layers 2–3** corners, textures → **Middle** wheels, barrels → **Deep** whole vehicles

Hierarchy: each layer combines the previous layer's patterns over a larger area. This was confirmed by visualising trained networks (Zeiler & Fergus, 2013).

Analogy: from individual sensor returns, to platoon formations, to a battalion's scheme of manoeuvre — each level of analysis builds on the one below.

A Complete CNN: 10-Class SAR Target Classifier

Layer	Operation	Output shape	Params
Input	SAR chip	$64 \times 64 \times 1$	0
Conv1	32 filters 3×3, same, ReLU	$64 \times 64 \times 32$	320
Pool1	max 2×2	$32 \times 32 \times 32$	0
Conv2	64 filters 3×3, same, ReLU	$32 \times 32 \times 64$	18 496
Pool2	max 2×2	$16 \times 16 \times 64$	0
Conv3	128 filters 3×3, same, ReLU	$16 \times 16 \times 128$	73 856
Pool3	max 2×2	$8 \times 8 \times 128$	0
GAP	global average pool	128	0
Dense	10 classes, softmax	10	1 290
Total			**93 962**

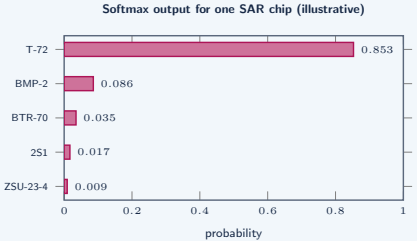
Parameter arithmetic

Conv1 $(9 \cdot 1 + 1) \cdot 32 = 320$; Conv2 $(9 \cdot 32 + 1) \cdot 64 = 18\,496$; Conv3 $(9 \cdot 64 + 1) \cdot 128 = 73\,856$; Dense $128 \cdot 10 + 10 = 1\,290$.

Pattern to notice

Spatial size halves ($64 \to 32 \to 16 \to 8$) while channels double ($32 \to 64 \to 128$): the network trades *where* for *what*.

GAP averages each 8×8 map to one number. For comparison, a single dense layer from the flattened 64×64 chip to 128 units would already need $524\,416$ parameters — over $5\times$ this entire CNN.



Logits $(4.1,\ 1.8,\ 0.9,\ 0.2,\ -0.4)$ for the top five classes $\rightarrow$ softmax.

Loss

Categorical cross-entropy: $L = -\ln P(\text{true class})$. If the chip is a T-72: $L = -\ln 0.853 = 0.159$.

Backpropagation through a conv layer

The same chain rule as before. Because one kernel is used at every position, its gradient is the *sum* of contributions from all positions:

$$\frac{\partial L}{\partial K_{m,n}} = \sum_{i,j} \delta_{i,j}\, x_{i+m,\, j+n}$$

— itself a convolution of the input with the error map. Max-pooling passes the gradient only to the winning pixel.

Training tricks

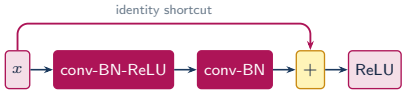
- **Data augmentation**: rotate, flip, shift, add noise — free extra examples
- **Batch normalisation**: stabilises activations, speeds training
- **Dropout**: randomly silences units to reduce over-fitting



The 2012 turning point

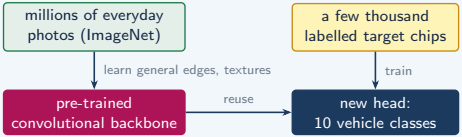
AlexNet cut ImageNet top-5 error to 15.3% against 26.2% for the next-best entry. Three years later ResNet ensembles reached 3.57%. Computer vision switched to deep learning almost overnight.

ResNet's residual block



Output $= F(x) + x$: the same skip connection used in every Transformer block.

Transfer Learning: When Labelled Military Data Is Scarce



Two options

- **Feature extraction**: freeze the backbone; train only the new head — fast, works with very little data
- **Fine-tuning**: also update the later backbone layers with a small learning rate — better when more data exist

Why it works: early-layer features (edges, corners, textures) are generic. Only the top layers are specific to the original task.

Caution for SAR: radar imagery looks very different from optical photos (speckle, shadows, layover). The benefit of optical pre-training is smaller and must be checked on held-out SAR data.

Synthetic training data from simulators and CAD models of vehicles is a common supplement — but beware the gap between simulated and real returns (distribution shift, Lecture 1).

Automatic target recognition (ATR)

Classifying vehicles in SAR chips. On the public MSTAR benchmark (10 vehicle classes) published CNNs report accuracies above 95% under standard test conditions.

Object detection in EO/IR imagery

Detectors such as Faster R-CNN and YOLO output a box and a class for every object — aircraft on aprons, ships in harbour, vehicles on roads.

Change detection and segmentation

Compare satellite images over time; label every pixel (road, building, crater) to update terrain databases for simulations.

Wargaming and simulation

The map is an image: CNN policies read unit positions, terrain and visibility layers as channels (the approach used in game-playing systems such as AlphaGo).

Beyond images

1-D CNNs on radar pulses, acoustic and RF signals (emitter identification); spectrograms treated as images.

"Standard conditions" means test vehicles similar to training ones. Accuracy drops sharply for new depression angles, configurations or camouflage — always ask how the test set was built.

Failure Modes Every Analyst Must Know

Adversarial examples. Tiny, invisible pixel changes can flip a prediction with high confidence (Goodfellow et al., 2014: a panda classified as a gibbon). Physical versions — printed patches — have fooled detectors in field tests.

Shortcut learning. A network may key on background, sensor artefacts or image borders instead of the target. A frequently retold (and probably apocryphal) story describes a "tank detector" that had learned the weather in the photos — the lesson is real even if the story is not.

Distribution shift. New sensor, season, terrain or decoys: performance can collapse with no warning, because softmax confidence stays high.

Checking what the CNN looks at

Saliency methods such as Grad-CAM colour the pixels that most influenced a decision. If the "hot" region is the background, not the vehicle, the model is not trustworthy — whatever its accuracy.

Mitigations: diverse training data, adversarial training, test sets from new conditions, and a human analyst confirming high-consequence calls.

CNNs and Transformers Compared

	CNN	Vision Transformer (ViT)
Basic operation	convolution: fixed local window	self-attention: every patch to every patch
Built-in assumption	nearby pixels matter; same pattern anywhere	none — relations learned from data
Data needed	works well on small/medium datasets	usually needs large datasets or pre-training
Long-range context	only through depth	from the first layer
Cost vs image size	linear in pixels	quadratic in number of patches
Shared ideas	dot-product similarity, ReLU-type non-linearities, residual connections, softmax + cross-entropy, backpropagation	

In practice many current vision systems are hybrids: a CNN front end for local features feeding attention layers for global context. For small, specialised defence datasets, CNNs remain a strong and economical first choice.

Common Misconceptions

"Engineers design the filters." All kernel values are learned by backpropagation; hand-made edge filters are only a teaching device.

"Pooling is learned." Max and average pooling have no parameters.

"A 3×3 filter looks at 9 numbers." It looks at $3 \times 3 \times C_{\text{in}}$ numbers — all input channels at once.

"CNNs are fully shift-invariant." Only approximately; small shifts can still change outputs, and rotations and scale changes need augmentation.

"High softmax = reliable." CNNs are often over-confident, especially on inputs unlike their training data.

Self-check: if you can reproduce the four steps of the worked example and the parameter table, you understand the core of every CNN.

Summary and Quick Check

CONVOLUTION

$$y_{i,j} \;=\; \sum_{m,n} K_{m,n}\, x_{i+m,j+n} \;+\; b$$

ACTIVATION AND POOLING

$$\mathrm{ReLU}(z) \;=\; \max(0, z), \qquad \text{maxpool: largest value per block}$$

OUTPUT SIZE

$$n_{\text{out}} \;=\; \lfloor (n_{\text{in}} \;+\; 2p \;-\; k)/s \rfloor \;+\; 1$$

PARAMETERS PER CONV LAYER

$$(k^2\, C_{\text{in}} \;+\; 1)\, C_{\text{out}}$$

OUTPUT

$$P(\text{class}) \;=\; \text{softmax}(W\ \text{GAP}(\text{features}) \;+\; b)$$

Quick check

1. In the worked example, what is the output at position (1,0) for K_1, and why?
2. A 128×128 input meets a 5×5 kernel with $p = 2$, $s = 2$. What is the output size?
3. How many parameters has a layer of 256 filters of 3×3 on 128 input channels?
4. Why does a uniform bright region give 0 with an edge kernel?
5. Name two ways an adversary could defeat a CNN-based target recogniser.

From pixels to words: models that read in context

- What a language model does: one token at a time
- Embeddings as coordinates on a "meaning map"
- Why one fixed vector per word fails — the "battery" problem
- Encoder and decoder: who reads, who writes
- Attention as a radio net: queries, keys, values — one computed by hand
- Multi-head attention, residuals, normalisation, cross-attention

Assignment 7: convolution, ReLU and pooling by hand + output-size formula + parameter counts for a CNN + receptive fields + an evaluation brief for a SAR recogniser; bonus: gradients through max-pooling and a shared kernel.

Preview: one word, many meanings

"The *battery* was out of charge" and "the *battery* opened fire" use the same word for a power cell and an artillery unit. A transformer gives each occurrence a different vector, built from the words around it.

A convolution scores a kernel against a patch with a dot product. Attention scores one word against another with the same dot product — but the "kernel" is chosen by the data.